LAB REPORT

Course Code : ICE 3146 **Course Title :** Microprocessor and Interfacing Laboratory
Lab Task No. : 06
Lab Task Topic: Generating LED Patterns and Displaying Numbers on a Seven Segment Display using Loops in the 8086 Microprocessor Trainer Kit in PC Mode.

No.	Evaluation Criteria with Marks	Put Tick (✓) Mark					Marks Obtained	Remarks
		Excellent	Good	Fair	Poor	Fail		
REPORT WRITING								
1	Theory & Objectives (2.5)							
2	Materials & Methods (5)							
3	Data Analysis & Discussion/Diagram (10)							
4	Precautions and Conclusion (2.5)							
5	Time Management (5)							
	TOTAL							
LAB PERFORMANCE								
1	Preparation for Lab (5)							
2	Experiment and Engagement (15)							
3	Communication (5)							
	TOTAL							

24th June, 2026 Date of Experiment	11th July, 2026 Date of Submission	______________ Teacher's Signature

Semester : Summer **Year :** 2026 **Level-Term :** L2-T2 **Section :** A2

Submitted to :
Sameer Khairul
Lecturer, Dept. of Information and Communiction Engineering
Daffodil International University

Submitted by :
Tanzim Ahamed
Reg.No. : 242-50-036
Dept. of Information and Communication Engineering
Daffodil International University

Title: Generating LED Patterns and Displaying Numbers on a Seven-Segment Display using Loops in the 8086 Microprocessor Trainer Kit in PC Mode.

Abstract:

This experiment was performed using the Intel 8086 Microprocessor Trainer Kit in PC Mode to generate LED patterns and display numbers on a seven-segment display by using assembly language instructions and repetitive program control. The main objective of this experiment was to understand how the 8086 microprocessor communicates with external output devices through I/O ports and how a sequence of display values can be shown continuously by using a data table, branching instructions, and a delay routine. In this experiment, the 8255 Programmable Peripheral Interface (PPI) was initialized first, and then predefined binary codes were sent one after another to Port A. As a result, the seven-segment display showed the output sequence 1, 1, 1, 2, 3, 4, and 9 repeatedly. This experiment helped to understand port addressing, hardware interfacing, delay generation, memory-based data access, and practical control of output devices using the 8086 microprocessor.

Introduction (Theory):

The Intel 8086 is a 16-bit microprocessor that can communicate with external hardware devices through Input/Output (I/O) ports. Devices such as LEDs and seven-segment displays are common output peripherals, and connecting these devices with the microprocessor is called interfacing. By using interfacing, the microprocessor can send data to external hardware and control its operation. In assembly language, the OUT instruction is used to transfer data from a register to an output port.

A seven-segment display is an electronic output device used to show decimal digits. It consists of seven individual segments, and different combinations of ON and OFF segments create different numbers. LEDs are also output devices, where different binary values produce different lighting patterns. In this experiment, the 8255 PPI was used as the interfacing device. Its control port and data ports were configured so that the connected seven-segment display and LED section could be driven by the trainer kit.

The program stores a sequence of binary codes in memory and reads them one by one through the SI register. Each value is then sent to Port A, which drives the seven-segment display. A delay subroutine keeps every output visible for a short time, and when the terminating value 00H is found, the program returns to the beginning of the data table. In this way, the display repeatedly shows the sequence 1, 1, 1, 2, 3, 4, and 9. The experiment demonstrates practical I/O operation, port initialization, memory addressing, repeated execution, and real-time hardware control using the 8086 microprocessor.

Material:

Hardware	Software
1. Intel 8086 Microprocessor Trainer Kit 2. Seven-Segment Display Interface 3. LED Interface Module / LED Board 4. Personal Computer (PC)	1. Microsoft Macro Assembler (MASM) 2. MDA-8086 / MDA-WinIDE8086 3. Text Editor (Notepad) 4. Command Prompt (CMD)

Method:

Code:

```
CODE SEGMENT
        ASSUME CS:CODE,DS:CODE,ES:CODE,SS:CODE
PPIC_C          EQU    1FH
PPIC            EQU    1DH
PPIB            EQU    1BH
PPIB            EQU    19H
                ORG    1000H
                MOV    AL,10000000B
                OUT    PPIC_C,AL
                MOV    AL,11110000B
                OUT    PPIB,AL
                MOV    AL,00000000B
                OUT    PPIC,AL
L2:             MOV    SI,OFFSET DATA
L1:             MOV    AL,BYTE PTR CS:[SI]
                CMP    AL,00H
                JE     L2
                OUT    PPIA,AL
                CALL TIMER
                INC    SI
                JMP    L1
                INT    3
TIMER:          MOV    CX,0
TIMER1:         NOP
                NOP
                NOP
                NOP
                LOOP   TIMER1
                RET
DATA:           DB     11111001B
                DB     11111001B
                DB     11111001B
                DB     10100100B
                DB     00110000B
                DB     10011001B
                DB     00010000B
                DB     00H
CODE            ENDS
END
```

<u>Execution (PC Mode):</u>

1. First, the assembly language program was written in Notepad and saved as .asm file.
2. Machine code execution using CMD-

    C:\MDA\8086\Asm8086>MASM.exe SUM26A16.asm

    Object filename [SUM26_1.OBJ]: SUM26A16
    Source listing [NUL.LST]: SUM26A16
    Cross-reference [NUL.CRF]:
        0 Warning Errors
        0 Severe Errors

1. The MDA (Microprocessor Development and Analysis) software was opened.
2. The SUM26A13.asm file was loaded using the CMD panel (LO).
3. Then in MDA terminal- LOD186 SUM26A16.ASM executed.
4. Then in MDA 8086 from FILE DOWNLOAD the SUM26A16.ABS file opened and Run executed.
5. The output and register values were observed through the simulator window and the Seven Segment Display shows the numbers according the code.

Result:

<u>Result for PC Mode</u>:

			
Initial condition of Display	Output for 11111001B	Output for 11111001B	Output for 11111001B
			
Output for 10100100B	Output for 00110000B	Output for 10011001B	Output for 00010000B

Fig 6.1: Seven Segment Display outputs for different Sequence of Code.

Observed Output Sequence:

Step	Displayed Number	Binary Code	Hex Value
1	1	11111001B	F9H
2	1	11111001B	F9H
3	1	11111001B	F9H
4	2	10100100B	A4H
5	3	00110000B	30H
6	4	10011001B	99H
7	9	00010000B	10H

Discussion:

In this experiment, LED patterns and the number sequence 1, 1, 1, 2, 3, 4, and 9 were generated on the 8086 Microprocessor Trainer Kit in PC Mode. The assembly program was written and assembled using MASM, and the resulting program was loaded and executed through the MDA-8086 software. This PC-based method simplified program development and reduced errors associated with manual machine-code entry.

Different binary codes were sent to the output ports using the OUT instruction. Each code produced a specific LED pattern or number on the seven-segment display. Loop instructions were used to repeat the required sequence efficiently, while a delay routine kept each output visible for a short period before the next value appeared.

The observed output matched the expected sequence, confirming that the port addresses, binary codes, and loop logic were correct. This experiment provided practical knowledge of output-port initialization, seven-segment encoding, loop control, delay generation, and hardware interfacing using 8086 assembly language..